AKS Backup & Restore: presenter script

Read along with the v7 slides. Each block matches one slide, in order. The animations run on their own, so just keep talking as the cards appear. Total time is about 27 minutes. The ten-minute restore time is indicative and depends on the size of the cluster.

Slide 1 | Cover 0:20

On screen: AKS Backup & Restore. The orbit graphic zooms in.

Good morning everyone. Today I will walk you through AKS Backup and Restore.

I will cover what it is, how we deploy it with our platform pipeline, and how you restore a cluster step by step in the Azure Portal.

Slide 2 | What is AKS Backup? 1:30

On screen: Three cards, then the amber card and three number tiles.

AKS Backup is an Azure service that takes scheduled backups of an AKS cluster and keeps them in our own subscription. It is a backup of the cluster itself, not just of the code that built it.

How does it work? A Backup extension runs inside the cluster. The Backup vault reaches the cluster through Trusted Access, and takes a backup every four hours.

What does it contain? The cluster state: the namespaces and the workloads, configuration, services and access rules inside them. In our setup, volume data is not included.

Look at the numbers at the bottom. A backup every 4 hours. Backups are kept 7 days in non-prod and 90 days in prod. And a restore takes about ten minutes. That ten minutes is indicative, and it depends on the size of the cluster.

The amber card sums it up: better than stateless pipelines. We restore the real cluster state to a chosen point in time.

Slide 3 | What is in the cluster state 1:15

On screen: Four cards. The last one, Not included, has an amber border.

Let me be precise about what is inside the backup.

Workloads: deployments, stateful sets, daemon sets, jobs and cron jobs, with their replica counts and settings.

Configuration: config maps, secrets, services and ingress, together with the namespaces they live in.

Access and custom resources: RBAC roles and bindings, service accounts, and custom resources with their definitions. Cluster-scoped resources are included too.

And the amber card is what is not included: volume data, because volume snapshots are off, and Azure resources outside the cluster. Our Terraform pipelines rebuild those.

At the bottom: our setup is a full cluster backup with nothing excluded.

Slide 4 | AKS Backup versus stateless pipelines 1:45

On screen: Rows appear one by one. The right column has teal borders and ticks.

Why do we take a backup instead of only re-running the pipelines? Let me go row by row.

What comes back: with pipelines, only what they last deployed. With AKS Backup, the real cluster state at the restore point.

Manual changes in the cluster: with pipelines they are lost, because they are in no pipeline. With AKS Backup they are captured.

Steps to recover: with pipelines you re-run each one, in the right order. With AKS Backup it is one restore from the Backup vault.

Time to restore: with pipelines it depends on how many have to run. With AKS Backup it is about ten minutes, again indicative.

Point in time: pipelines give you the latest committed version only. AKS Backup gives you a restore point every four hours, kept for seven days in non-prod and ninety days in prod.

Test without risk: we can restore to the -rst cluster and the original is never touched.

And the key message at the bottom: the two work together. Terraform builds the cluster and its Azure resources, and AKS Backup brings back what runs inside it.

Slide 5 | What we will cover next 0:30

On screen: Six numbered tiles.

Here is the plan for the rest of the session.

One, how AKS Backup works. Two, our architecture. Three, what we built in code. Four, permissions. Five, restoring in the Portal. And six, how to verify and the common traps.

Slide 6 | How AKS Backup works 1:30

On screen: Three tall cards, then the amber-outlined note.

Three ideas to remember.

First, the Backup extension. It is a component installed inside the cluster. The Backup vault talks to the cluster only through this extension, for both backup and restore.

Second, Trusted Access. The vault reaches the cluster through Trusted Access, with a least-privilege role. No broad admin rights are handed to the vault.

Third, the operational tier. Backups stay in our own subscription, in the backup storage account and the snapshot resource group. This is the tier we use.

One more point from the note: it backs up cluster resources. Volume snapshots are off, so volume data is not included.

Slide 7 | Our architecture 1:45

On screen: Read left to right. Numbered arrows appear in order.

Let me walk through the diagram from left to right.

On the left is the Backup vault with its backup policy. Arrow number one is Trusted Access: the vault reaches the original cluster, which has the Backup extension installed.

Arrow number two is the backup data. The extension writes the backup into the storage account and its container.

Under the storage account you can see the private endpoint. The cloud team creates it. If there is no private endpoint, the storage account cannot talk to the cluster, and backups fail.

Now the amber path, number three: restore. From the vault, we restore into the restore cluster, the one with the -rst suffix. This is an alternate-cluster restore, so the original cluster is never touched.

Slide 8 | What the AKS platform pipeline creates 1:30

On screen: Six cards, then the file path at the bottom.

One file, backup dot tf, builds the whole backup setup. Here is what the pipeline creates.

The storage account and container: this is where backup data lives. Public network access is off, which is why the private endpoint is needed. Versioning is on.

The Backup extension, installed on the cluster. It tells the cluster which storage account and container to write backups to.

The Backup vault: zone redundant, soft delete always on, immutable, and protected by a delete lock.

The Backup policy: a backup every four hours, kept seven days in non-prod and ninety days in prod.

The Backup instance ties together the cluster, the vault and the policy. Full cluster backup, nothing excluded.

And access and roles: Trusted Access from the vault to the cluster, plus the roles that the vault, the extension and the cluster identities need.

Slide 9 | How backup.tf is driven by conf.yaml 1:15

On screen: Three boxes with arrows, then two cards.

No names are hard-coded. backup dot tf reads lists from conf dot yaml and builds one resource for every entry.

So conf dot yaml holds our settings: lists of storage accounts, vaults, clusters and policies. backup dot tf reads each list. And the result is the Azure resources: storage, vault,

extension, policy, instance and roles.

You may ask: what is a local? A local is a named helper value that Terraform works out once and then reuses. For example, backup dot tf turns the storage account list into a lookup by name. Locals are internal helpers. They are not settings you pass in.

Why it matters: add an entry to a list in conf dot yaml and the next pipeline run creates the matching resources. Retention is seven days everywhere except prd, which keeps ninety days.

Slide 10 | What conf.yaml contains for backup 1:30

On screen: Five cards, then the -rst note.

Conf dot yaml has five blocks for backup. For each one you see what you put in, and what it creates.

Storage account: you give a name and a container name, and it creates the storage account and blob container.

Backup vault: a vault name, and it creates the vault with a delete lock.

Backup extension: the cluster, the storage account and the container, and it creates the extension on that cluster.

Backup policy: a name and the vault. Note that the interval and retention are set in backup dot tf, not here.

Backup instance: the cluster and the policy. It creates the instance, plus Trusted Access and the role assignments for that cluster.

A note on the -rst cluster. Every -rst entry is commented out in the dev conf dot yaml. It gets an extension but no backup instance, so it is a restore target and is not backed up itself.

Slide 11 | How we deploy AKS Backup 1:30

On screen: Four numbered steps, then the verify checklist.

Deploying it takes four steps, all through code and the pipeline.

Step one: update conf dot yaml with the backup settings for the environment you need: storage account, vault, policy and cluster.

Step two: create a pull request and merge it to main after review.

Step three: run the aksplatform release pipeline for that environment. It creates the backup resources.

Step four: verify the resources.

Use this checklist: the backup vault is created, the extension is on the cluster, the backup instance is configured, the first backup job completed, and the private endpoint has been

requested from the cloud team.

Slide 12 | What we did in workload onboarding 1:45

On screen: Three cards, then the amber result bar.

We also made three changes in workload onboarding, so that the -rst cluster works without touching the original.

Phase one: clusters with -rst- in the name are left out of the Prometheus alert list, so alert rule names no longer clash. The cluster-names output that phase two uses is the filtered list too, which fixed the workspace routing failure. The old hard-coded exclusion list was removed.

Phase two: a hard-coded environment mapping was replaced by detection from conf dot yaml. Any cluster with -rst- in its name is treated as a restore target. The code looks it up in the same resource group and reads its OIDC issuer.

The module: for each workload component that uses workload identity, we create an extra federated credential for the -rst cluster. Same identity, same subject, different issuer. Each cluster has its own issuer, so without it pods on the -rst cluster fail the token exchange and stick in Init. The original cluster's credentials are never changed.

The result: the phase one plan showed only an output change and no resource changes for the original cluster.

Slide 13 | Restore options at a glance 1:00

On screen: Four cards. Alternate cluster is highlighted.

Now to restoring. AKS Backup is flexible about what and where you restore.

A full restore brings back everything that was backed up: the cluster resources. Volume data is not part of our backups.

An item-level restore lets you pick only what you need: namespaces, labels or specific resource kinds.

You can restore to the original cluster. Existing resources with the same name must be deleted first, or patched.

Or you restore to an alternate cluster. This is how we validate, with the -rst cluster.

Slide 14 | Permissions behind backup and restore 1:30

On screen: Six role cards, then two notes.

Permissions. This is what backup dot tf grants.

The AKS cluster identity gets Contributor on the snapshot resource group.

The Backup vault gets Reader on the AKS cluster. It also gets Reader, Disk Snapshot Contributor and Data Operator for Managed Disks on the snapshot resource group, and

Storage Blob Data Contributor on the backup storage account.

The Backup extension identity gets Storage Account Contributor on the backup storage account.

And the Trusted Access binding from the vault to the cluster uses the backup-operator role.

Two notes. backup dot tf creates these for clusters that have a backup instance, so only the original cluster. The -rst cluster has an extension but no backup instance, so Validate will ask you to grant them.

And worth checking: Microsoft's restore guidance lists Storage Blob Data Contributor for the extension identity, but backup dot tf grants it Storage Account Contributor.

Slide 15 | Restoring a cluster (section slide) 0:15

On screen: Large restore icon in the orbit.

Now let us do it. Restoring a cluster in the Azure Portal, step by step.

Slide 16 | The restore in six moves 1:00

On screen: Six numbered nodes along a line.

Here is the whole restore in six moves. Everything happens from the Backup vault of the environment.

One, open the Backup vault. Two, select the backup instance. Three, click Restore and pick the restore point. Four, select the target cluster. Five, select the resources. Six, validate, review and restore.

Use the vault that belongs to the environment you are restoring: dev, sit, uat and so on. The restore needs the Backup extension installed on the target cluster, and Trusted Access enabled between the vault and that cluster.

Slide 17 | Restore in the Portal: finding the backup 1:15

On screen: Three step cards, then the Portal path pills appear one by one.

Moves one to three. Finding the backup.

One: in the Azure Portal, search Backup vaults and open the vault for your environment.

Two: open Backup instances and select the AKS backup instance.

Three: click Restore, go to the Restore point tab, and choose the point in time you want to restore.

The row of pills is the Portal path: Azure Portal, Backup vaults, your vault, Backup instances, the AKS backup instance, Restore, and then Restore point in amber.

Slide 18 | Restore in the Portal: running the restore 1:30

On screen: Three cards. The last one has an amber border.

Moves four to six. Running the restore.

Four: on Restore parameters, click Select Kubernetes Service and choose the target cluster, for example the -rst cluster.

Five: choose which namespaces and resources to restore.

Six: click Validate. For the -rst cluster, backup dot tf does not create Trusted Access or the role assignments, so expect Validate to ask for them. Grant what it asks for, then validate again until it passes. Then review the summary, click Restore, and follow progress under Backup jobs in the vault.

Slide 19 | Choosing what to restore 1:15

On screen: Six filter tiles on the left, two option cards on the right.

On the left are the filters for an item-level restore. Namespaces: pick from the backed-up list. Cluster-scoped resources: tick to include them. Labels: restore only resources with matching labels. API groups and kinds: restore specific types, such as Deployment. Namespace mapping: restore into a different namespace, which is created if missing. And, optional, a Resource Modifier config map can change properties during restore.

On the right is what happens if a resource already exists. Skip is the default: the existing resource is left as it is, so delete it first if you want it restored. Patch updates the mutable fields of the existing resource, for example the replica count.

Slide 20 | After the restore: verify the result 1:00

On screen: Terminal panel with four commands, two cards, amber bar.

After the restore, verify the result.

On the terminal: kubectl get pods with the namespace, kubectl get deployments with the namespace, kubectl get pods with the dash capital A flag to see all namespaces, and kubectl get ns to see the namespaces. You want namespaces back, pods running and services present.

Also follow the restore job under Backup jobs in the vault until it shows as completed, and confirm the restored workloads start and respond as expected.

And the rule in the amber bar: always test the restored workloads before calling the restore a success.

Slide 21 | Things that trip people up 1:30

On screen: Six cards. Same names clash is highlighted.

Read these before your first restore.

Existing resources: backups do not delete and recreate resources, so delete the existing resource before restoring it in place.

Namespaces: a namespace that does not exist is created during restore. Use namespace mapping to restore next to the original.

Same region: backups are operational tier, so restore into the same region. Cross-region restore needs vault tier, which is not part of this setup.

No auto scaling: restore does not add nodes. Make sure the target cluster has enough capacity.

Same names clash: resources with the same name are skipped by default. Choose Patch if you want them updated.

And prerequisites: the Backup extension must be installed on the target cluster, and Trusted Access enabled.

Slide 22 | Our rollout, end to end 1:15

On screen: Six nodes along a line, then the amber note.

Here is our rollout from start to finish.

One, conf dot yaml: the backup settings for the environment. Two, a pull request, reviewed and merged to main. Three, the aksplatform pipeline, which creates storage, vault, extension, policy, instance and roles.

Four, the private endpoint, which the cloud team adds for the storage account. Five, workload onboarding, which picks up the -rst cluster. And six, verify: the vault, the extension, the instance and the first backup.

The -rst cluster is optional. Skip it if you only need backups.

Slide 23 | Summary and resources 0:45

On screen: Four numbered takeaways, links, and Questions.

To wrap up, four takeaways.

One: the vault reaches the cluster through the Backup extension and Trusted Access.

Two: restore starts in the Backup vault of the environment: Restore, restore point, target, resources, validate.

Three: grant the roles the validation asks for, then restore.

Four: verify with kubectl and check the restore job.

The Microsoft Learn links are on the slide for the restore guide, the backup overview and the backup concepts. Thank you. Any questions?